OpenCode 接口设计文档（v1.0）

OpenCode 主平台 ↔ 调度 Agent ↔ 子 Agent（Numina、AlphaEvolve...）

无废话、全可用、前后端都能直接照着写。

基于 OpenCode 的数学研究多智能体平台

接口设计文档（v1.0）

一、接口总体设计原则

1. **全部为 JSON 结构**
2. **异步任务模式**（数学计算耗时）
3. 统一请求/响应格式
4. 调度 Agent 完全控制流程，平台只做转发与环境
5. 子 Agent 无状态、可插拔
6. 支持多轮迭代、上下文传递

二、统一数据结构

1. 通用请求结构

Code block

```
{
  "task_id": "唯一任务ID",
  "session_id": "研究会话ID（长期上下文）",
  "input": "用户输入：公式/猜想/思路/片段",
  "context": "历史上下文（推导链、结果、状态）",
  "metadata": {}
}
```

2. 通用响应结构

Code block

```
{
  "code": 0,
  "msg": "ok",
  "task_id": "任务ID",
  "status": "running / finished / failed",
  "result": {},
  "context": "更新后的上下文",
  "next_action": "continue / rerun / finish"
}
```

三、核心接口（共 6 个，全部你需要的）

1. 用户创建研究任务

接口： `POST /api/task/create`

作用： 用户输入数学内容，创建研究任务

请求：

Code block

```
{
  "input": "对以下公式进行结构探索与推广：f(x) = ...",
  "metadata": {}
}
```

响应：

Code block

```
{
  "task_id": "task_123456",
  "session_id": "sess_7890",
  "status": "created"
}
```

2. OpenCode → 调度 Agent：启动研究流程

接口：`POST /api/scheduler/run`

作用：主平台把任务交给调度 Agent，由它全权控制

请求：

Code block

```
{
  "task_id": "task_123456",
  "session_id": "sess_7890",
  "input": "用户原始输入",
  "context": {}
}
```

响应：

Code block

```
{
  "status": "running",
  "next_action": "call_agents",
  "agent_list": ["numina", "alpha_evolve"],
  "context": "调度后新上下文"
}
```

关键点：

调度 Agent **自己决定**调用谁、怎么调用。

3. 调度 Agent → 子 Agent：执行数学能力

3.1 调用 Numina（符号计算）

接口：`POST /api/agent/numina/exec`

请求：

Code block

```
{
  "task_id": "task_123456",
  "session_id": "sess_7890",
```

```
  "goal": "化简/符号推导/代数变形",
  "input": "数学表达式",
  "context": {}
}
```

响应：

Code block

```
{
  "result": "化简后的公式",
  "steps": ["步骤1","步骤2"],
  "status": "success"
}
```

3.2 调用 AlphaEvolve（进化构造/探索）

接口： `POST /api/agent/alpha_evolve/exec`

请求：

Code block

```
{
  "task_id": "task_123456",
  "goal": "结构探索/构造反例/推广猜想",
  "input": "原始结构",
  "context": {}
}
```

响应：

Code block

```
{
  "candidates": ["结构A","结构B"],
  "counter_examples": [],
  "insights": "发现规律..."
}
```

3.3 通用子 Agent 调用格式（可扩展任意数学Agent）

接口： `POST /api/agent/{agent_name}/exec`

4. 子 Agent → 调度 Agent：返回结果

接口：`POST /api/scheduler/collect`

作用：所有子 Agent 计算完，把结果回传给调度 Agent

请求：

Code block

```
{
  "task_id": "task_123456",
  "agent_name": "numina",
  "result": {},
  "status": "success"
}
```

响应：

Code block

```
{
  "status": "collecting",
  "next": "continue / verify / finish"
}
```

5. 调度 Agent → OpenCode：返回最终研究结果

接口：`POST /api/task/finish`

作用：调度 Agent 认为研究完成，提交最终结论

请求：

Code block

```
{
  "task_id": "task_123456",
  "status": "finished",
  "conclusion": "最终研究结论",
  "derivation": ["推导步骤"],
  "insights": "研究洞察",
  "context": "最终上下文"
}
```

响应：

Code block

```
{
  "code": 0,
  "msg": "研究任务已完成"
}
```

6. 查询任务状态（轮询/长连接）

接口： `GET /api/task/status?task_id=xxx`

响应：

Code block

```
{
  "status": "running / finished",
  "progress": "30%",
  "current_stage": "调度Agent决策中",
  "result": {}
}
```

四、上下文结构（数学研究专用）

Code block

```
{
  "original_input": "用户输入",
  "goal": "研究目标",
  "derivations": ["推导历史"],
  "agent_results": {
    "numina": {},
    "alpha_evolve": {}
  },
  "hypothesis": "当前猜想",
  "verified": true/false,
  "counter_examples": [],
  "next_idea": "下一步思路"
}
```

五、接口调用流程（极简版）

1. 用户创建任务
2. OpenCode 交给调度 Agent
3. 调度 Agent **自主选子 Agent**
4. 调用 Numina / AlphaEvolve
5. 子 Agent 计算 → 回传调度 Agent
6. 调度 Agent 整合、判断、迭代
7. 完成 → 返回 OpenCode → 返回用户

六、为什么这个接口设计适合你？

- **调度 Agent 完全控制一切**，无硬编码
- 子 Agent 即插即用
- 支持**非问题、开放式数学研究**
- 支持多轮、迭代、探索
- 轻量、易实现、易扩展
- 完全匹配你的架构：OpenCode + 调度Agent + 多子Agent

你要我继续给你：

1. **调度 Agent 的完整 Prompt（数学研究专用）**
2. **数据库表结构设计**
3. **部署架构图**
4. **简易演示流程（文字版）**

告诉我，我直接给你写好。

（注：文档部分内容可能由 AI 生成）